

2.1 Debugging

Sam has just spent an hour and a half in the lab tracking down a bug that turned out to be a small typo. What a waste of time! There are so many better things he could have been doing with that time (such as trying to get a date with that cute girl in his computer class named Sue). If only there was some way to get his program to tell him where the problems were, then this whole process would be much simpler!

Objectives

By the end of this class, you will be able to:

- Create asserts to catch many of the most common programmer problems.
- Use `#define` to move constants to the top of a program.
- Use `#ifdef` to create debug code in order to test a function.
- Write a driver program to verify the correctness of a function.
- Create stub functions to make an outline of a large program.

Prerequisites

Before reading this section, please make sure you are able to:

- Create a function in C++ (Chapter 1.4).
- Convert a logic problem into a Boolean expression (Chapter 1.5).
- Pass data into a function using both pass-by-value and pass-by-reference (Chapter 1.4).
- Measure the cohesion level of a function (Chapter 2.0).
- Measure the degree of coupling between functions (Chapter 2.0).
- Create a map of a program using structure charts (Chapter 2.0).

Asserts

When writing a program, we often make a ton of assumptions. We assume that a function was able to perform its task correctly; we assume the parameters in a function are set up correctly; and we assume our data structures are correctly configured. A diligent programmer would check all these assumptions to make sure his code is robust. Unfortunately, the vast majority of these checks are redundant and, to make matters worse, can be a drain on performance. A method is needed to allow a programmer to state all his assumptions, get notified when the assumptions are violated, and have the checks not influence the speed or stability of the customer's program. Assertions are designed to fill this need.

An assert is a check placed in a program representing an assumption the developer thinks is always true. In other words, the developer does not believe the assumption will ever be proven false and, if it does, definitely wants to be notified. An assert is expressed as a Boolean expression where the true evaluation indicates the assumption proved correct and the false evaluation indicates violation of the assumption. Asserts are evaluated at run-time verifying the integrity of assumptions with each execution of the check.

An assert is said to fire when, during the execution of the program, the Boolean expression evaluates to `false`. In most cases, the firing of an assert triggers termination of the program. Typically the assert will tell the programmer where the assert is located (program name, file name, function, and line number) as well as the assumption that was violated.

Assertions have several purposes:

- **Identify logical errors.** While writing a program, assertions can be helpful for the developer to identify errors in the program due to invalid assumptions. Though many of these can be found through more thorough investigation of the algorithm, the use of assertions can be a time saver.
- **Find special-case bugs.** Testers can help find assumption violations while testing the product because their copy of the software has the asserts turned on. Typically, developers love this class of bugs because the assert will tell the developer where to start looking for the cause of the bug.
- **Highlight integration issues.** During component integration activities or when enhancements are being made, well-constructed assertions can help prevent problems and speed development time. This is the case because the asserts can inform the programmer of the assumptions the code makes regarding the input parameters

Assertions are not designed for:

- **User-initiated error handling.** The user should never see an assert fire. Asserts are designed to detect internal errors, not invalid input provided by the user.
- **File errors.** Like user-errors, a program must gracefully recover from file errors without asserts firing.

Syntax

Asserts in C++ are in the `cassert` library. You can include asserts with:

```
#include <cassert>
```

Since asserts are simply C++ statements (more precisely, they are function calls), they can be put in just about any location in the code. The following is an example assert ensuring the value `income` is not negative:

```
assert(income >= 0);
```

If this assert were in a file called `budget.cpp` as part of a program called `a.out` in the function called `computeTithing`, then the following output would appear if the assumption proved to be invalid:

```
a.out: budget.cpp:164: float computeTithing(float income): Assertion `income >= 0'
failed.
Aborted
```

It is important that the client never sees a build of the product containing asserts. Fortunately, it is easy to remove all the asserts in a product by defining the `NDEBUG` macro. Since asserts are defined with pre-processor directives, the `NDEBUG` macro will effectively remove all assert code from the product at compilation time. This can be achieved with the following compiler switch:

```
g++ -DNDEBUG file.cpp
```



Sue's Tips

The three most common places to put asserts are:

1. At the top of a function to verify that the passed parameters are valid.
2. Just after a function is called, to ensure that the called function worked as expected.
3. Whenever any assumption is made.

Example 2.1 – Asserts	
Demo	This example will demonstrate how to add asserts to an existing program. This will include how to brainstorm of where bugs might exist, common checkpoints where asserts may reside, and how to interpret the messages that asserts give us.
Problem	Given a program to compute how much tithing an individual should pay given an income, add asserts to catch common bugs. <pre>What is the income? <u>100</u> Tithe for \$100 is \$10</pre>
Solution	The first part is to include the assert library: <pre>#include <cassert></pre> Next, we will add asserts to the computeTithe() function. <pre>float computeTithing(float income) { assert(income >= 0.00); // this only works for positive income // compute the tithing float tithe = income * 0.10; assert(tithe >= 0.00); // The Lord doesn't owe us, right? assert(income > tithe); // 10% should be less than 100%, right? // return the answer return tithe; }</pre> Observe how the asserts both validate the input (<code>income >= 0</code>) and perform sanity checks that the resulting output is not invalid (<code>tithe >= 0.0</code> as well as <code>income > tithe</code>). The first assert is designed to make sure the function is called correctly. The second is to make sure the math was performed correctly..
Challenge	As a challenge, add asserts to your Project 1 solution. Would any of these have caught bugs you ran into when you were writing your code?
See Also	The complete solution is available at 2-1-asserts.cpp or: <pre>/home/cs124/examples/2-1-asserts.cpp</pre> 

#define

The `#define` mechanism is a way to get the compiler to do a search-replace through your file before the program is compiled. This is useful for values that never change (like π). We will also use this to do more advanced things. An example of `#define` in action is:

Before expansion	After expansion
<pre>include <iostream> using namespace std; #define PI 3.14159 /***** * MAIN * Simple program to * demonstrate #define *****/ int main() { cout << "The value of pi is " << PI << endl; return 0; }</pre>	<pre>include <iostream> using namespace std; /***** * MAIN * Simple program to * demonstrate #define *****/ int main() { cout << "The value of pi is " << 3.14159 << endl; return 0; }</pre>

Note that `#defines` are always ALL_CAPS according to our style guidelines.

Observe how the value in the `#define` can be used much like a variable. In many ways, it is like a variable with one important exception: it can't vary. In other words, the value represented by the `#define` is guaranteed to not change during the course of the program.

Sam's Corner

There is another way to make a variable that does not change: a constant variable:

```
const float PI = 3.14159;
```

Observe how the syntax is similar to any other variable declaration, including using the assignment operator with the semicolon. It is important to realize that this is not a global variable: the `const` modifier guarantees that the value in the variable does not change. Global variables are only dangerous because they can change in an unpredictable way



A few common uses of `#defines` are:

- **Constants:** Values that never change. Through the use of `#defines`, it is much easier to verify that all instances of the constant are the same in the program. We don't want to have more than one version of π for example.
- **Static file names:** Some file names, such as configuration files, are always in the same location. By using a `#define` for the file name, it is easy to see which files the program accesses and to ensure that all the parts of the code refer to the same file.

We can also put parameters in `#define` macros. Here, the syntax is similar to that of a function:

```
#define NEGATIVE(x) (-x)           // also ALL_CAPS
```

Again, this will expand just before compilation just like the non-parameter `#define` does. Consider the following code:

Before expansion	After expansion
<pre>#include <iostream> using namespace std; #define ADD_TEN(x) (x + 10) /***** * MAIN * #define expansion demo *****/ int main() { int value = 5; cout << value << " + 10 = " << ADD_TEN(value) << endl; return 0; }</pre>	<pre>#include <iostream> using namespace std; /***** * MAIN * #define expansion demo *****/ int main() { int value = 5; cout << value << " + 10 = " << (value + 10) << endl; return 0; }</pre>

For more information about the `#define` pre-processor directive, please see:

[#define](#)

#ifdef

Another pre-processor directive (along with `#define` and `#include`) is the `#ifdef`. The `#ifdef` preprocessor directive tells the compiler to optionally compile some code depending on the state of a condition. This makes it possible to have some code appear only in a Debug version of the program. Consider the following code:

```

/*****
 * COMPUTE TAX
 * Compute the monthly tax
 *****/
float computeTax(float incomeMonthly)
{
    float incomeYearly = incomeMonthly * 12.0;

#ifdef DEBUG
    cout << "incomeYearly == "          // the code between #ifdef and #endif
          << incomeYearly << endl;      // only gets compiled if the
                                          // DEBUG macro is defined
#endif // DEBUG

    float taxYearly;

    // tax code
    ...

#ifdef DEBUG
    cout << "taxYearly == "              // observe how we format the output so we
          << taxYearly << endl;           // can tell which variable we are
                                          // looking at in the output stream
#endif // DEBUG

    return taxYearly / 12.0;
}

```

In this example, we have debug code displaying the values of key variables. Note that we don't always want this code to execute; test bed will certainly complain about the unexpected output. Instead, we only want the

code to run when we are trying to fix a problem. The `#ifdef` mechanism allows this to occur. We can “turn on” the debug code with:

```
#define DEBUG
```

If this appears before the `#ifdefs`, then all the code will be included in the compilation and the `couts` will work as one expects. This allows us to have two versions in a single code file: the ship version containing code only for the customer to see, and the debug version containing tons of extra code to validate everything.



Sam's Corner

An `#define` can also be turned on at compilation time without ever touching the source code. We do this by telling the compiler we want the macro defined:

```
g++ -D<MacroName>
```

For example, if you want to turn on the `DEBUG` macro without using `#define DEBUG`, this can be accomplished with:

```
g++ -DDEBUG file.cpp
```

As an exercise, please take a close look at (`/home/cs124/examples/2-1-debugOutput.cpp`). Please see the following link for more detail on how `#ifdef` works.

[#ifdef](#)



Sam's Corner

The aforementioned `#ifdef` technique to display debug code can be tedious to write.

Fortunately there is a more convenient way to do this. First, start with the following macro at the top of your program:

```
#ifdef DEBUG
#define Debug(x)  x
#else
#define Debug(x)
#endif
```

This macro actually does something quite clever. If `DEBUG` is defined in your program, then anything inside the `debug()` statement is executed. If `DEBUG` is not defined, then nothing is executed. Consider the following code:

```
void function(int input)
{
    Debug(cout << "input == " << input << endl);
}
```

If `DEBUG` is defined, then the above is expanded to:

```
void function(int input)
{
    cout << "input == " << input << endl;
}
```

If `DEBUG` is not defined, we get:

```
void function(int input)
{
}
```

Example 2.1 – Debug Output

Demo

This example will demonstrate how to add COUT statements to get some insight into how the program is behaving. It will do this by utilizing `#define` directives, `#ifdef` directives, and asserts.

Problem

Write a program to compute an individual's pay taking into account time-and-a-half overtime.

```
What is your hourly wage? 12  
How many hours did you work? 39.5  
Pay: $ 474.00
```

From this example, insert debug code to help discover the location of bugs.

```
What is your hourly wage? 12  
How many hours did you work? 39.5  
main: hourlyWage: 12  
main: hoursWorked: 39.5  
computePay(12.00, 39.50)  
computePay: Regular rate  
Pay: $ 474.00
```

The first thing to do is to add a mechanism to easily put debug code in the program.

```
#ifdef DEBUG  
#define Debug(x) x  
#else  
#define Debug(x)  
#endif // !DEBUG
```

Now, if `DEBUG` is not defined, none of the code in `Debug()` gets executed. If it is defined, then we can get all our debug output. The `computePay()` function with `Debug()` code is:

```
float computePay(float hourlyWage, float hoursWorked)  
{  
    Debug(cout << "computePay(" << hourlyWage << ", " << hoursWorked << ")\n");  
    float pay;  
  
    // regular rate  
    if (hoursWorked < CAP)  
    {  
        Debug(cout << "computePay: Regular rate\n");  
        pay = hoursWorked * hourlyWage;                // regular rate  
    }  
  
    // overtime rate  
    else  
    {  
        Debug(cout << "computePay: Overtime\n");  
        pay = (CAP * hourlyWage) +                    // first 40 normal  
              ((hoursWorked - CAP) * (hourlyWage * OVERTIME)); // balance overtime  
    }  
    return pay;  
}
```

See Also

The complete solution is available at [2-1-debugOutput.cpp](#) or:

```
/home/cs124/examples/2-1-debugOutput.cpp
```



Driver programs

Drivers are special programs designed to test a given function. This is an exceedingly important part of the programming process. An aerospace engineer would never put an untested engine in an airplane. He would instead mount the engine on a testing harness and run it through the paces. Only after exhaustive testing would he feel confident enough to put the engine in the airplane. We should also treat new functions with skepticism. When we validate functions before integrating them into the larger program, it is far easier to localize problems. After the function has been validated, then we can safely copy it to the project. Typically drivers consist of just the function `main()` and the function to be tested. Consider, for example, the prototype for the function `computePay()`:

```
float computePay(float hourlyWage, float hoursWorked);
```

A driver program for `computePay()` might be:

```

/*****
 * MAIN
 * Simple driver for computePay()
 *****/
int main()
{
    float wage;
    cout << "wage: ";                // get the data as quickly as possible
    cin >> wage;

    float hours;
    cout << "hours: ";                // again, just the simplest prompt
    cin >> hours;

    cout << "computePay("
        << "hourlyWage = " << wage << ", " // show what was passed
        << "hoursWorked = " << hours
        << ") == "
        << computePay(wage, hours)         // show what was returned
        << endl;

    return 0;
}

```

Observe how the driver program is just a bare-bones program whose only purpose is to prompt the user for the data to pass to the function and to display the results. When you use the driver-program development methodology, you:

1. Start with a blank file. The only thing this program will do is test your function.
2. Write the function. As long as the coupling is loose, this should not be too complex.
3. Create a `main()` that only calls your function. This is typically done in three steps:
 - a. First call your function with the simplest possible data.
 - b. If your function requires any parameters, create simple `cin` statements in `main()` to fetch that data directly from the user.
 - c. If your function returns something, display the results directly on the screen so it is easy to verify how the function responded to input.
4. Test your function with a variety of input. Start with simple input and work to more complex scenarios.

Example 2.1 – Driver

Demo

This example will demonstrate how to write a simple driver program to test a function we used in Project 1: `computeTax()`.

Solution

The driver program exists entirely in `main()`. We start with the prototype of the function we are testing:

```
double computeTax(double incomeMonthly);
```

There will be two parts: fetching the data from the user for the function parameters, and displaying the output of the function.

```
/* *****  
 * MAIN  
 *   A simple driver program for computeTax()  
 * ***** */  
int main()  
{  
    // get the income  
    double income;           // the inputs to the function being  
    cout << "Income: ";      //   tested is gathered directly from  
    cin  >> income;          //   the user and sent to the function  
  
    // call the function and display the results  
    cout << "computeTax(" << income << ") == " // what we are sending...  
         << computeTax(income)                 // what the output is  
         << endl;  
  
    return 0;  
}
```

Driver programs are very streamlined and simple. Once we have tested our function, we can safely throw them away.

Challenge

As a challenge, write a driver program for `computeTithe()` from your Project 1 code.

See Also

The complete solution is available at [2-1-driver.cpp](#) or:

```
/home/cs124/examples/2-1-driver.cpp
```



This process works well when you are in the development phase of the project. You can also use the driver-program technique when you are in the testing and debugging phase of the project. This can be accomplished by modifying our `main()` to be a driver for any function in the program. Recall the `computeTax()` function from Project 1. We might think we have worked out all the bugs of the functions before they were integrated together. When running the program, however, it becomes apparent that something is broken.

Consider the following `main()` from Project 1 after it has been modified to test `computeTithing()`. Note how we use a `return` statement to ensure only the top part of the function is executed.

```

/*****
 * MAIN
 * Keep track of a monthly budget
 *****/
int main()
{
    double incomeTest = getIncome();           // use the get function or a cin
    cout << computeTithing(incomeTest) << endl; // simple display of the output
    return 0;                                  // return ensures we exit here and
                                              // only test computeTithing()
                                              // rest of main below here

    // instructions
    cout << "This program keeps track of your monthly budget\n";
    cout << "Please enter the following:\n";
    // prompt for the various data
    double income      = getIncome();
    double budgetLiving = getBudgetLiving();
    double actualLiving = getActualLiving();
    double actualTax    = getActualTax();
    double actualTithing = getActualTithing();
    double actualOther  = getActualOther();

    // display the results
    display(income, budgetLiving, actualTax, actualTithing,
            actualLiving, actualOther);
    return 0;
}

```

The driver program technique has been used for almost all the assignments we have done this semester.

Stub functions

A stub is a placeholder for a forthcoming replacement promising to be more complete. In the context of designing and building a program, a stub is a tool enabling us to put a placeholder for all the functions in our structure chart without getting bogged down with how the functions will work. In other words, stubs allow us to:

- **Get Started:** Stubs allow us to get the design from the Structure Chart into our code before we have figured out how to implement the functions themselves. This helps answer the question “how do I start on this project?”
- **Figure out data flow.** Because stubs include the parameters passed between functions, you can model information flow early in the development process.
- **Always have your program compile.** A program completely stubbed-out will compile even though it does not do anything yet. Then, as you implement individual functions, any compile errors you encounter are localized to the individual function you just implemented.

Consider the computeTax function from Project 1. The following would be an example of a stub:

```
float computeTax(float income)
{
    // stub for now...
    return 0.0;
}
```

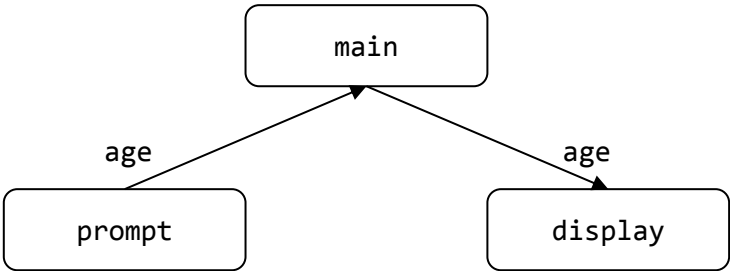
Example 2.1 – Stub Functions

Demo

This example will demonstrate how to turn a structure chart into stub functions.

Problem

Write stub functions for the following structure chart.



Solution

The stubbed version of this program would be:

```
int prompt()                // don't bother with the function comment block
{                            // with stubbed functions. We will do them later
    return 0;                // make sure to return some value because the return
}                            // type is not void in this function

void display(int age)        // all the parameters need to be present in the stub
{                            // even if the body of the functions are empty.
}

int main()                  // make sure the stubs call all the children
{                            // functions so we can make sure the data flow
    display(prompt());       // works correctly. This should enable us to
    return 0;                // implement the functions in any order we choose
}
```

Observe how the stubbed functions consist of:

- **Prototypes:** the function name, return type, and parameters.
- **Empty body:** except for a return statement, the body is mostly empty.
- **Called functions:** include code to call the child functions. It is OK to use dummy parameters if none are known.

Challenge

As a challenge, try to stub Project 1. A sample solution is available at [2-1-stubbed.cpp](#) or:

```
/home/cs124/examples/2-1-stubbed.cpp
```

Problem 1

Which of the following most clearly illustrates the concept of coupling?

- The parameters passed to a function
- The task that the function performs
- The subdivision of components in the function or program
- The degree in which a function can be used for different purposes

Please see page 117 for a hint.

Problem 2

Identify the type of coupling that the following function exhibits:

```
float lastGrade = 0.0;

float getGPA()
{
    cout << "What is your grade? ";
    cin >> lastGrade;

    return lastGrade;
};
```

Answer:

Please see page 117 for a hint.

Problem 3

Create a stub for the following function:

```
void displayIncome(float income)
{
    // configure output to display money
    cout.setf(ios::fixed);
    cout.setf(ios::showpoint);
    cout.precision(2);

    // display the income
    cout << "Your income is: $"
        << income
        << endl;
};
```

Answer:

Please see page 140 for a hint.

Problem 4

Create a stub for the following function:

```
float getIncome()
{
    float income;

    // prompt
    cout << "Please enter your income: ";
    cin >> income;

    return income;
}
```

Answer:

Please see page 140 for a hint.

Problem 5

Create a driver program for the following function:

```
float sqrt(float value);
```

Answer:

Please see page 138 for a hint.

Problem 6

Create a driver program for the following function:

```
void displayTable(int numDaysInMonth, int offset);
```

Answer:

Please see page 138 for a hint.

Problem 7

Create an assert to verify the following variable is within the expected range:

```
float gpa
```

Answer:

Please see page 132 for a hint.

Problem 9

What asserts would you add to the beginning of the following function:

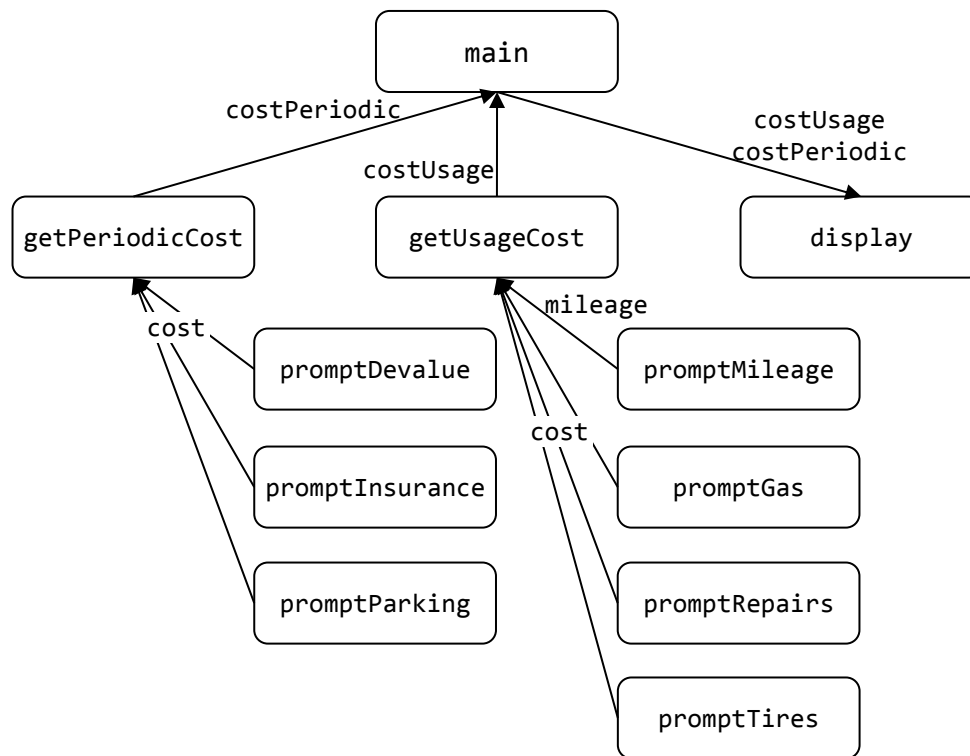
```
void displayDate(int day, int month, int year);
```

Answer:

Please see page 49 for a hint.

Assignment 2.1

Sue wants to write a program to help her determine how much money she is spending on her car. Specifically, she wants to know how much she spends per day having the car sit in her driveway and how much she spends per mile driving it. While working through this problem, she came up with the following structure chart:



Please create stub functions for all the functions in Sue's program. In other words, write a program to stub out every function represented in the above structure chart. If a function calls another function (ex: `getPeriodicCost()` calls `promptParking()`), then make sure that function call is in the stub. Finally, make sure all the parameters and return values from the structure chart are represented in the stub functions.

One final note: you do not need to have function headers for each individual function.

Please:

- Create stub functions for all the functions mentioned in the structure chart.
- If you were able to do this, then enter the following code in the function `display()`:

```
cout << "Success\n";
```

- Use the following testbed:

```
testBed cs124/assign21 assign21.cpp
```

- Submit to Assignment 21

Please see page 140 for a hint.